

Resumen de Agentes Connect-4

Fundamentos de Inteligencia Artificial 2026.1 | Universidad de La Sabana

Sara Nicol Zuluaga Parra – Axel Daniel Beodya Quintero

Agente 1: Monte Carlo First Visit

1. Idea principal del agente

El agente implementa Monte Carlo First Visit (MCFV), un método de aprendizaje por refuerzo basado en episodios completos. En cada decisión, el agente simula varias partidas aleatorias (rollouts) desde el estado actual hasta el final del juego y usa esos resultados para estimar el valor de cada movimiento candidato. El conocimiento acumulado se guarda en una Q-table que se actualiza promediando los retornos obtenidos, pero solo desde la primera visita de cada par (estado, acción) en el episodio, lo que evita sesgar el estimador con repeticiones dentro del mismo episodio.

Diferenciación respecto al grupo: A diferencia de agentes puramente reactivos o de Q-Learning tabular con actualizaciones por paso, MCFV aprende del resultado global de la partida, no de transiciones individuales. Esto lo hace especialmente adecuado para Connect-4, donde la recompensa solo ocurre al ganar o perder.

2. Versiones del agente

Version 1 (con bloqueo): Incluye verificación explícita de victorias inminentes del oponente antes de evaluar los rollouts. Si el rival puede ganar en el siguiente turno, bloquea inmediatamente (use_blocking=True). Parámetros base: num_rollouts=5, epsilon=0.1.

Version 2 (sin bloqueo): Igual que V1 pero sin el módulo de bloqueo (use_blocking=False). La selección recae completamente en los rollouts y la Q-table. Permite medir el aporte real de la sub-estrategia de bloqueo sobre el desempeño.

3. Análisis de desempeño

3a. Desempeño contra el jugador aleatorio (300 partidas por escenario)

Version	Color	Victorias	Derrotas	Empates	Tasa Victoria
V1 (con bloqueo)	Rojo	~264	~0	~36	~88%
V1 (con bloqueo)	Amarillo	~243	~0	~57	~81%
V2 (sin bloqueo)	Rojo	~222	~15	~63	~74%
V2 (sin bloqueo)	Amarillo	~198	~24	~78	~66%

Conclusion: V1 nunca pierde contra el aleatorio en ningún color. El bloqueo aporta aproximadamente 14 puntos porcentuales de mejora, confirmando que es una sub-estrategia de alto impacto.

3b. Autodesempeño: V1 vs V1 (300 partidas)

Jugador	Victorias	Derrotas	Empates	Tasa Victoria
---------	-----------	----------	---------	---------------

Rojo (-1)	~162	~108	~30	~54%
Amarillo (1)	~108	~162	~30	~36%

Conclusion: Hay un sesgo de color: rojo tiene ventaja (~54%) porque mueve primero. El agente no logra neutralizar completamente esa asimetría posicional, lo que es esperable sin conocimiento perfecto del juego.

3c. Efecto de num_rollouts (variable numerica) — vs Aleatorio como Rojo (150 partidas)

num_rollouts	Tasa Victoria	Tiempo/Partida	Balance
1	~68%	~12 ms	Insuficiente
3	~78%	~28 ms	Aceptable
5	~85%	~47 ms	Optimo
10	~88%	~92 ms	Bueno pero costoso
20	~90%	~185 ms	Retorno marginal

Conclusion: La tasa de victoria satura alrededor de num_rollouts=10. El punto optimo es num_rollouts=5: ofrece ~85% de victorias con menos de la mitad del tiempo de cómputo que num_rollouts=20.

4. Propuesta de mejora

Cuello de botella 1 — Alta varianza en rollouts aleatorios: Con num_rollouts bajo, las estimaciones fluctuan mucho porque los movimientos del rollout son completamente aleatorios. Esto causa decisiones suboptimas especialmente en fases finales del juego. **Mejora propuesta:** Rollouts guiados aplicando una heurística simple dentro del rollout (jugar victoria inmediata si existe, luego aleatorio). Esto reduce la varianza sin aumentar el numero de simulaciones.

Cuello de botella 2 — Explosion del espacio de estados: Connect-4 tiene ~4.5 trillones de estados posibles. La Q-table solo cubre los visitados, sin capacidad de generalizar a tableros similares. **Mejora propuesta:** Reemplazar la Q-table por una red neuronal pequeña (DQN) que tome el tablero como input y estime el valor Q por columna, permitiendo generalización entre tableros similares.

Agente 2: OhYes — MCTS

1. Idea principal del agente

OhYes utiliza Monte-Carlo Tree Search (MCTS) En cada turno sigue esta lógica de prioridad:

1. Si existe jugada ganadora, jugarla.
2. Si el rival amenaza ganar, bloquearlo.
3. Ejecutar MCTS con UCB1 para explorar el árbol de búsqueda.

El parámetro clave es `n_simulations` (simulaciones MCTS por turno).

diferenciación respecto al grupo: Mientras otros agentes del grupo aplican MCTS puro o búsqueda con función de evaluación estática, OhYes integra detección táctica $O(n)$ antes del MCTS, lo que mejora la calidad de decisión con presupuesto reducido.

Enlace al Código: ver branch del estudiante en el repositorio del grupo

2. Relacion con el curso

OhYes es una instancia directa de Trial-Based Online Policy. Improbamente. En cada estado s_t , se ejecuta un subproceso MCTS (trials internos) para estimar $q(s_t, a)$ localmente y luego se selecciona la mejor acción. Las simulaciones internas son descartadas y no contaminan la política exterior. UCB1 implementa el balance exploracion-explotacion dentro del árbol (tree policy).

3. Analisis del agente

3a. vs Jugador Aleatorio (100 partidas)

Color	OhYes gana	Aleatorio gana	Empate
Rojo (-1)	~86%	~12%	~2%
Amarillo (1)	~82%	~16%	~2%

3b. Impacto de `n_simulations` vs Aleatorio

Con `n_sim=10` el agente gana ~58%; con `n_sim=120` sube a ~84%. El rendimiento se estabiliza alrededor de `n_sim=120-200` (retorno decreciente).

3c. Autodesempenho: OhYes vs OhYes

El primer jugador (Rojo) gana ~55% de las partidas, consistente con la ventaja teórica del primer jugador en Connect-4. El resultado cercano al 50% valida la simetría de la estrategia.

4. Propuesta de mejora

Cuello de botella 1 — Rollout aleatorio: La política por defecto en simulación es aleatoria. Reemplazarla por detección de amenazas activas (threat-count heuristic) mejorar la calidad de los estimadores de valor sin mayor costo computacional.

Cuello de botella 2 — Arbol efimero: El árbol MCTS se descarta cada turno. Reutilizar el subárbol del movimiento elegido (*tree reuse*) conservaría información y duplicaría la profundidad efectiva de búsqueda.

Cuello de botella 3 — Trampas dobles: El agente no detecta jugadas que generan dos amenazas simultaneas. Anadir detección de *fork threats* reduciría derrotas en finales de juego.

análisis completo: entrega.ipynb